

SRS



Session: 2021-2025

Submitted By:

Muhammad Moaz (Frontend, Documentation Developer) 2021-CS-101
Maryam Siddique (Backend, Design Developer) 2021-CS-180
Fiza Azhar (Full Stack, Documentation Developer) 2021-CS-193
Alishba Javed (Team Lead, Full Stack Developer) 2021-CS-205

Subject:

Final Year Project

Submitted To:

Core Committee

Department of Computer Science
University of Engineering and Technology, Lahore

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Technologies To Be Used	1
1.3.1	Mobile App Development	1
1.3.2	Backend Development	1
1.3.3	Database Management	1
1.3.4	Web Development	1
1.3.5	Notification System	1
1.3.6	Offline Functionality	2
1.3.7	API and Integration	2
1.3.8	Security and Compliance	2
1.4	Definitions, Acronyms, and Abbreviations	2
2	Overall Description	3
2.1	Product Perspective	3
2.2	Product Functions	3
2.2.1	Image Diagnosis	3
2.2.2	RASA-powered Chatbot Interaction	3
2.2.3	Offline Capability	3
2.2.4	Feedback Mechanism	4
2.3	User Classes and Characteristics	4
2.4	Operating Environment	4
2.5	Design and Implementation Constraints	4
2.6	Assumptions and Dependencies	5
3	Specific Requirements	6
3.1	Functional Requirements	6
3.2	Non-Functional Requirements	6
3.3	Data Requirements	7
3.4	Hardware Requirements	7
4	System Architecture	8
5	System Interfaces	8
5.1	Mobile Application	8
5.2	Back-end API	8
5.3	Database	8
5.3.1	Software Interface	8
5.3.2	User Interfaces	9
6	Validation and Verification	10
6.1	Validation:	10
6.2	Verification:	10

7	Design	11
7.1	Use Case Diagram	11
7.2	Class Diagram	12
7.3	Activity Diagram	13
7.4	Sequence Diagram	14
7.5	State Diagram	15
7.6	Entity Relationship Diagram	16
7.7	Data Flow Diagram	17
7.7.1	DFD0	17
7.7.2	DFD1	18
7.7.3	DFD2	19
7.8	Component Diagram	20
7.9	Deployment Diagram	21
7.10	Context Diagram	22
8	Future Scope	23
8.1	Enhanced Diagnostic Capabilities	23
8.2	Artificial Intelligence	23
8.3	Data Analytics	23
8.4	Mobile and Web Expansion	23
8.5	Collaboration with Veterinary Networks	23
8.6	User Community Features	23
8.7	Regulatory Compliance	23
8.8	Sustainability Practices	23
8.9	Financial Integration	23
8.10	Augmented Reality (AR)	24
9	Conclusion	25
10	Appendices	26
10.1	Appendix A: Glossary of Terms	26
10.2	Appendix B: API Specifications	26
10.3	Appendix C: Detailed Algorithms	26
11	References	27

Declaration

We, the undersigned, hereby give our consent to the project described above.

Main Advisor:

Prof. Dr. Muhammad Shoaib

Signature:

Co-Advisor:

Ms. Abqa Javeed

Signature:

Verification

We, the undersigned faculty members, verify that we have reviewed the project mentioned above
and find it satisfactory.

Faculty Member 1:
Signature:

Faculty Member 2:
Signature:

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for V-DermX, a mobile-based application designed to detect and diagnose Lumpy Skin Disease (LSD) in Cattle. The app uses machine learning (ML) and deep learning (DL) models, specifically InceptionV3 and MobileNetV2, to identify skin diseases through image recognition. It also provides disease consultation via a chatbot and functions offline, targeting rural users with limited access to veterinary services.

1.2 Scope

The V-DermX application allows users to:

- Capture and upload images of animals for diagnosis.
- Receive accurate disease identification and treatment recommendations.
- Interact with an AI-powered chatbot for consultation.
- Work offline, syncing data when a connection is available.

The system is aimed at farmers, veterinarians, and rural users with limited or unavailable veterinary services.

1.3 Technologies To Be Used

1.3.1 Mobile App Development

React Native is used to create a cross-platform mobile app for Android, ensuring a seamless user experience.

1.3.2 Backend Development

Nest.js, a powerful Node.js framework, is employed for building the server-side logic. It offers modular architecture and easy maintainability.

1.3.3 Database Management

MongoDB, a NoSQL database, is used for flexible and scalable data storage. It is optimized for handling large datasets and ensuring real-time updates.

1.3.4 Web Development

The intro website is developed using the MERN stack (MongoDB, Express.js, React, Node.js), ensuring dynamic content and smooth user interaction.

1.3.5 Notification System

Socket.io is implemented to handle real-time notifications and updates, providing instant feedback and alerts to users.

1.3.6 Offline Functionality

Service Workers enable offline capabilities, allowing the app to function without an active internet connection and sync data once reconnected.

1.3.7 API and Integration

APIs are developed for integrating external services, enabling smooth data flow between the app and other systems.

1.3.8 Security and Compliance

Security features such as encryption and authentication mechanisms are incorporated to protect user data and ensure system integrity.

1.4 Definitions, Acronyms, and Abbreviations

- **LSD:** Lumpy Skin Disease
- **AI:** Artificial Intelligence
- **ML:** Machine Learning
- **DL:** Deep Learning
- **CNN:** Convolutional Neural Network
- **InceptionV3:** A deep learning architecture used for image classification
- **MobileNetV2:** A lightweight neural network architecture

2 Overall Description

2.1 Product Perspective

The V-DermX app consists of:

- A mobile app for capturing images of animals and providing disease detection using pre-trained AI models (InceptionV3, MobileNetV2).
- An AI chatbot that assists users with questions related to animal diseases and treatments.
- Offline functionality where users can still capture images, interact with the chatbot, and receive diagnoses without a network connection, syncing data when online.

The app integrates:

- A front-end mobile interface (Android) for user interaction.
- A back-end server for AI processing, chatbot queries, and data synchronization.
- Cloud/Local storage for saving offline data.

2.2 Product Functions

2.2.1 Image Diagnosis

The user can capture or upload an image of the animal's skin condition, and the AI model will diagnose potential cases of Lumpy Skin Disease.

Image Analysis The system will utilize advanced deep learning models, specifically InceptionV3 and MobileNetV2, to process and analyze the uploaded images. The primary objective is to detect lumpy skin disease (LSD) and provide an initial diagnosis.

Diagnosis Output Based on the analysis, the app will present users with an on-screen diagnosis. This diagnosis will include potential disease identification and suggestions for treatment or prevention, helping users to take immediate action.

2.2.2 RASA-powered Chatbot Interaction

Users can consult a RASA-based chatbot integrated into the application. The chatbot will leverage Natural Language Processing (NLP) techniques to interpret user queries related to lumpy skin disease, including symptoms, prevention, and treatment options.

Knowledge-Based Access The chatbot will draw information from a comprehensive disease knowledge base. This allows users to receive relevant and accurate responses to their inquiries, ensuring they are well-informed about the disease and its management.

2.2.3 Offline Capability

Users can operate the app offline, and data syncs with the server when an internet connection is restored.

Service Worker Integration The app will utilize Service Workers to ensure seamless offline functionality. Users will be able to access essential features such as image uploads, chatbot interaction, and viewing previously accessed content without requiring an internet connection.

Local Data Storage In offline mode, any captured images, consultation history, or feedback will be stored locally using IndexedDB. The app will cache images and relevant data to prevent data loss.

Automatic Syncing Once internet connectivity is restored, the app will automatically sync all locally stored data, including images, diagnostic results, and chatbot interactions. This ensures that all relevant information is submitted to the backend for further analysis and record-keeping.

2.2.4 Feedback Mechanism

After receiving a diagnosis or completing a chatbot consultation, users will have the opportunity to provide feedback on their experience. This will allow users to rate the accuracy of the diagnosis or the quality of chatbot responses.

2.3 User Classes and Characteristics

- **Farmers:** Users with minimal technical skills who rely on the app for quick and simple guidance on identifying animal health issues. Their primary need is to receive clear, actionable advice and contact information for nearby veterinarians for further assistance.
- **Veterinarians (Contact Information):** Although veterinarians won't use the app directly, their contact details are crucial for farmers seeking professional help. The app will list available veterinarians, ensuring that users can easily reach out for expert diagnosis and treatment options.
- **Rural Users:** Users in remote areas with limited internet access and restricted access to veterinary services. Offline functionality will be essential to provide these users with basic diagnostic tools, advice, and veterinarian contact information even when offline.

2.4 Operating Environment

The V-DermX app will function on mobile platforms (Android), designed for environments with limited or intermittent internet access. It will need access to the device camera for image capture and local storage for offline usage.

2.5 Design and Implementation Constraints

- **Accuracy:** The AI models should maintain an accuracy level of 92.7% (as per InceptionV3) for diagnosing lumpy skin disease.
- **Performance:** The app must provide results within 10 seconds of image submission.
- **Data Limitations:** Availability of sufficient and relevant datasets for training and improving the DL models.

2.6 Assumptions and Dependencies

- The system assumes users have devices capable of capturing images.
- The models rely on sufficient lighting and clear image captures to ensure accurate disease detection.
- Internet availability for syncing and feedback submission.

3 Specific Requirements

3.1 Functional Requirements

1. Image Diagnosis:

- Users can upload or capture images of animal skin conditions.
- The system analyzes the image using InceptionV3 and MobileNetV2 to detect lumpy skin disease.
- The app provides an on-screen diagnosis, with potential disease identification and treatment suggestions.

2. Chatbot Consultation:

- Users can interact with a chatbot to ask questions related to the disease, its symptoms, prevention, and treatments.
- The chatbot responds with information from the disease knowledge base, powered by Natural Language Processing (NLP) techniques.

3. Offline Support:

- The app can operate without an internet connection, storing images and data locally.
- Once connected to the internet, the app automatically syncs stored data and submits it for analysis.

4. User Feedback:

- After diagnosis or consultation, users can provide feedback regarding the service quality.
- Feedback is stored in the backend and used for system improvements.

3.2 Non-Functional Requirements

1. Performance:

- Image processing should complete within 10 seconds for a good user experience.
- The chatbot response time must not exceed 2-3 seconds for a typical query.

2. Reliability:

- The system must be able to handle intermittent connectivity and continue functioning offline.

3. Usability:

- The user interface must be intuitive, ensuring easy navigation for non-technical users, such as farmers.

4. Security:

- All user data, including images and chat logs, must be securely stored and transmitted using encryption techniques.
- **User Authentication:** Secure authentication methods, such as JWT (JSON Web Token) are used to verify users, and passwords are hashed using Bcrypt for additional protection.
- **Data Encryption:** Encryption is applied both in transit (using HTTPS) and at rest, ensuring that all data remains private and secure.
- **Role-based Access Control (RBAC):** Permissions are assigned based on user roles to restrict access to sensitive features and data.
- **Audit Logging:** Security events are logged for auditing purposes to monitor and track unauthorized access attempts.

5. Scalability:

- The system must scale to handle multiple users across different rural locations, with varying network strengths.

6. Portability:

- The app must be compatible with Android devices and should work on low-end smartphones to accommodate rural users.

3.3 Data Requirements

- **Image Data:** Captured images will be processed by the AI model for disease detection and stored locally for offline use.
- **Chat Data:** User queries and responses will be logged for improvement of the chatbot's knowledge base.
- **Feedback Data:** Collected after every diagnosis or chatbot interaction.

3.4 Hardware Requirements

- **Camera Quality:** Users need mobile devices with a good-quality camera (8 MP or higher) to capture clear images of animal skin conditions for accurate disease detection.
- **Processing Power:** Devices should have sufficient processing power (quad-core processor or higher) to handle image analysis and real-time chatbot interactions without lag.
- **Storage:** Mobile devices should have adequate storage (at least 1 GB free space) to store images locally, especially when offline functionality is used.
- **RAM:** A minimum of 2 GB of RAM is recommended for smooth app operation, especially when running image diagnosis and the chatbot simultaneously.
- **Offline Data Handling:** Devices should support Service Workers to store images and data offline, ensuring the app's ability to function without internet access and later sync data when online.

4 System Architecture

The system architecture consists of:

- **Mobile App (Android):** Captures images, provides results, and interacts with the chatbot.
- **Back-end Server:** Hosts AI models (InceptionV3, MobileNetV2) for disease detection, and serves as the backend for the chatbot and data synchronization.
- **Database:** Stores user data, feedback, and syncs local offline data with the backend server.

5 System Interfaces

5.1 Mobile Application

Interfaces with the device camera for image capture.

5.2 Back-end API

Interfaces for uploading images, receiving diagnoses, interacting with the chatbot, and syncing offline data.

5.3 Database

Interfaces for storing feedback, diagnostic results, and disease-related data.

5.3.1 Software Interface

The Vet Derm-X app interfaces with several key software components to deliver its functionality, including:

Database Management System: The app integrates with MongoDB, a secure NoSQL database, to store and retrieve user data, diagnostic images, chatbot interactions, and system logs. This ensures data integrity, scalability, and accessibility for real-time processing.

Web Services: The app uses web services to handle online disease detection, data syncing, and chatbot interaction. Users can access disease information, prevention tips, and consultation services through mobile browsers and the app, ensuring 24/7 access from any device.

Notification Services: Socket.io is implemented to manage real-time notifications. This allows the app to deliver instant updates about diagnosis results, treatment reminders, and alerts to users across devices.

Machine Learning Models: The system interfaces with InceptionV3 and MobileNetV2 machine learning models for image analysis, enabling the detection of lumpy skin disease from uploaded images.

Third-Party APIs: The app can integrate with external services through APIs for additional functionality, such as providing enhanced disease knowledge, veterinary networks, and real-time data updates from external databases.

Offline Functionality: The app leverages Service Workers for offline support, ensuring that users can store images and data locally and sync them with the backend when internet connectivity is restored.

5.3.2 User Interfaces

This section describes the visual layout and components of the system. The system must be user-friendly and designed to optimize the user experience across different devices.

- **Splash Screen:** Displays the Vet Derm-X logo and app name for 3-5 seconds. Shows a loading indicator if needed.
- **Sign Up Screen:** Allows new users to create an account in the app.
- **Login Screen:** Allows returning users to log into their accounts.
- **Home Screen:** Quick access buttons: *Scan Animal*, *Disease Info*, *Chatbot*.
- **Scan/Detection Screen:** Camera access and image upload options. Includes a *Submit for Detection* button and a loading indicator.
- **Result Screen:** Displays scan results to determine whether the disease was detected or not. Option to access vet dermatologist chatbot for advice.
- **Chatbot Screen:** Chat interface with text input and predefined options. Bot responses provide medical advice and preventive measures.
- **Medication/Precaution Screen:** List of medications with dosage and usage. Includes preventive steps and an option to save/download info.
- **Profile Screen:** The user can view or edit his/her profile, change the password, and sign out.
- **Help & Support Screen:** Provides FAQs, contact support, and app tutorial options.

6 Validation and Verification

6.1 Validation:

- Validate AI model performance by measuring the accuracy and speed of the diagnosis.
- Validate the chatbot responses for correctness and relevance.

6.2 Verification:

- Test the offline capability by simulating a loss of network and ensuring that data syncs correctly once connectivity is restored.
- Conduct usability testing with real users (farmers, veterinarians) to ensure ease of use.

7 Design

7.1 Use Case Diagram

The use case diagram provides a high-level overview of the system's actors and their interactions with the system's functionalities.

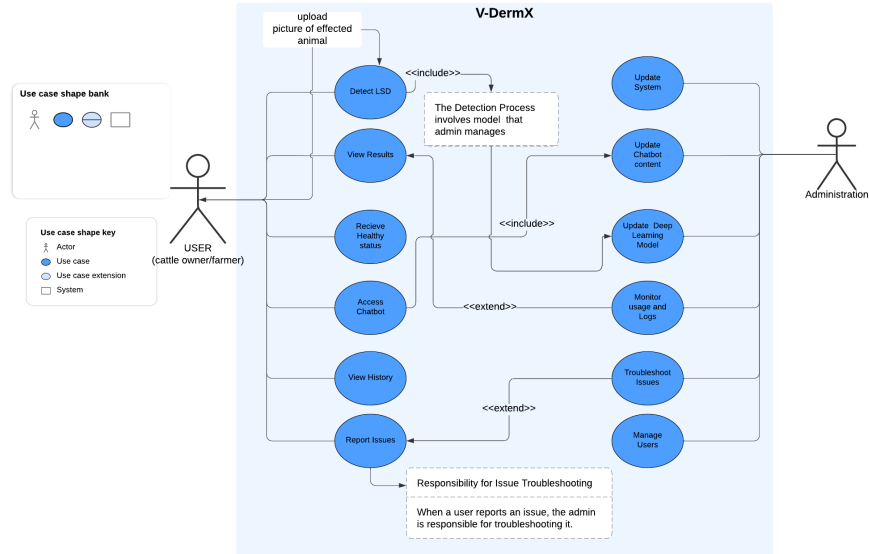


Figure 1: Use Case Diagram illustrating the interactions between the users and system functionalities.

7.2 Class Diagram

The class diagram details the structure of the system by showing its classes, attributes, methods, and relationships between the classes.

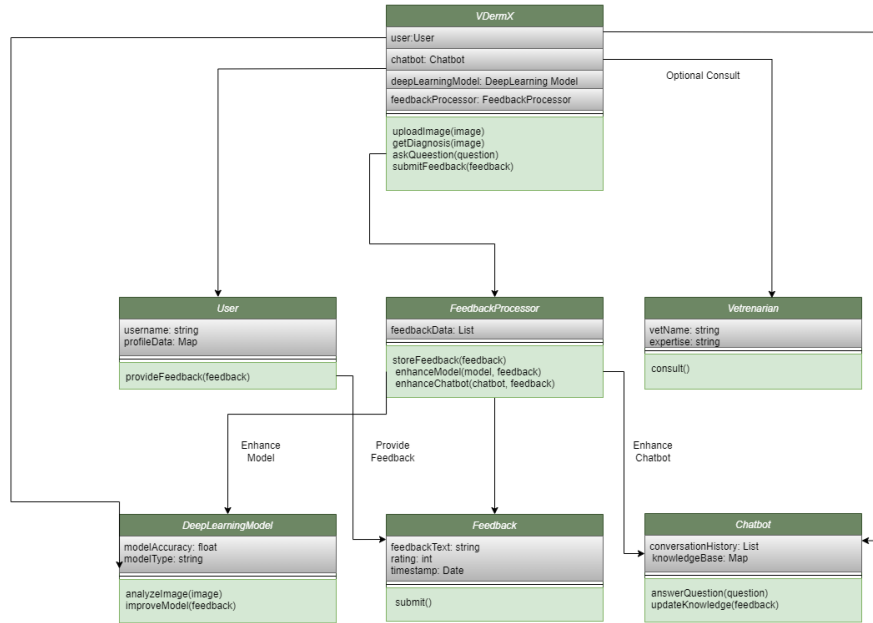


Figure 2: Class Diagram depicting the relationships between the various system classes.

7.3 Activity Diagram

The activity diagram shows the flow of control or data within the system, representing the steps of a particular process.

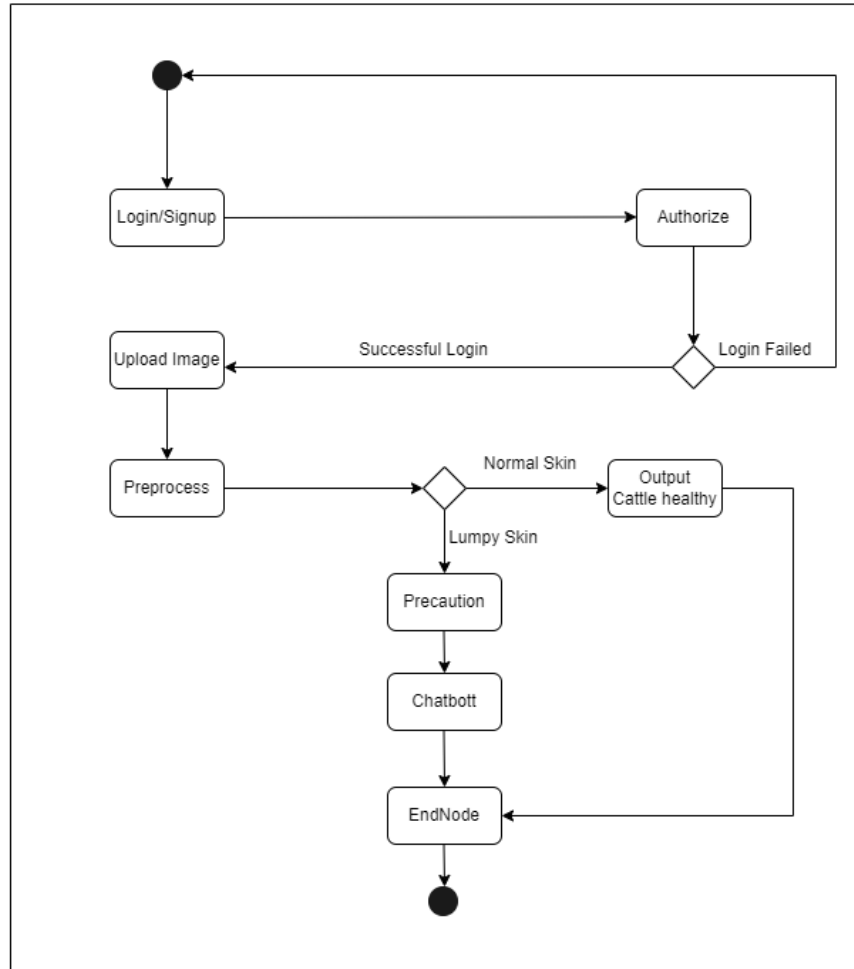


Figure 3: Activity Diagram demonstrating the flow of control for a particular process in the system.

7.4 Sequence Diagram

The sequence diagram represents the time-based interaction between objects, showing the order in which the processes occur.

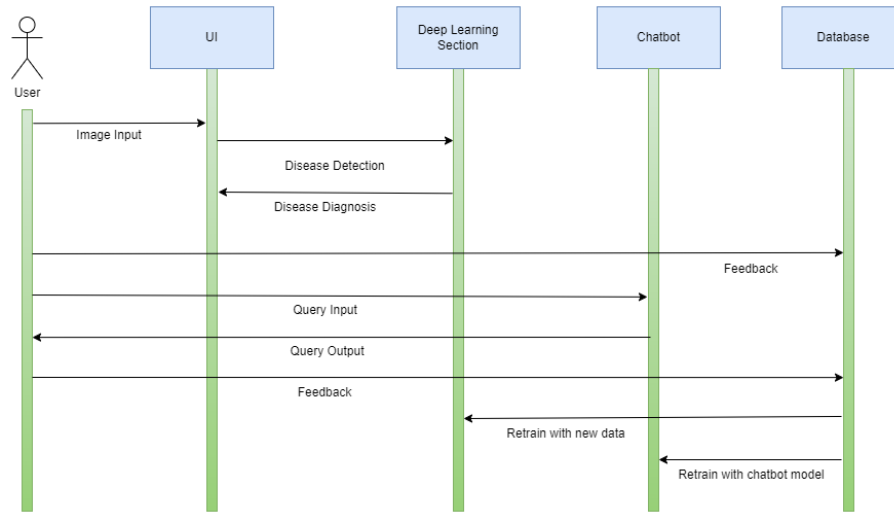


Figure 4: Sequence Diagram outlining the sequence of interactions between system components.

7.5 State Diagram

The state diagram depicts the states and transitions that an object goes through during its lifecycle in the system.

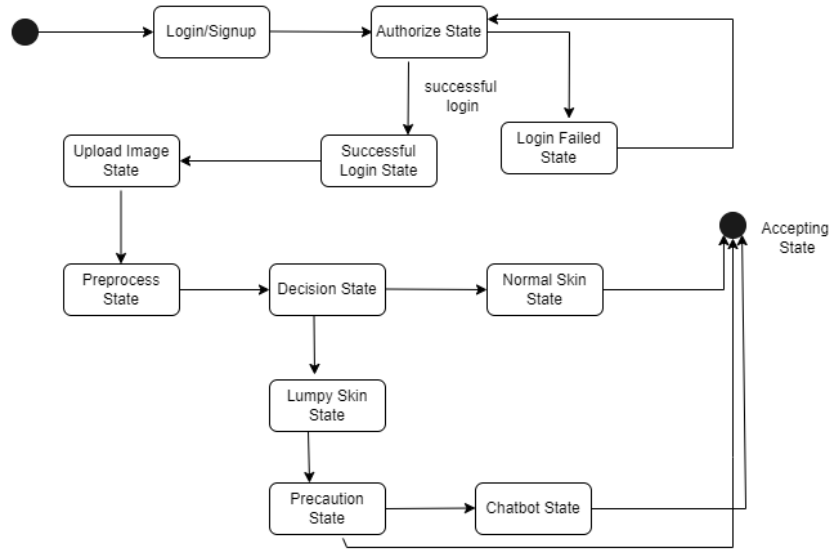


Figure 5: State Diagram showing the states an object transitions through in its lifecycle.

7.6 Entity Relationship Diagram

The entity relationship diagram (ERD) models the data relationships within the system, focusing on entities and their attributes.

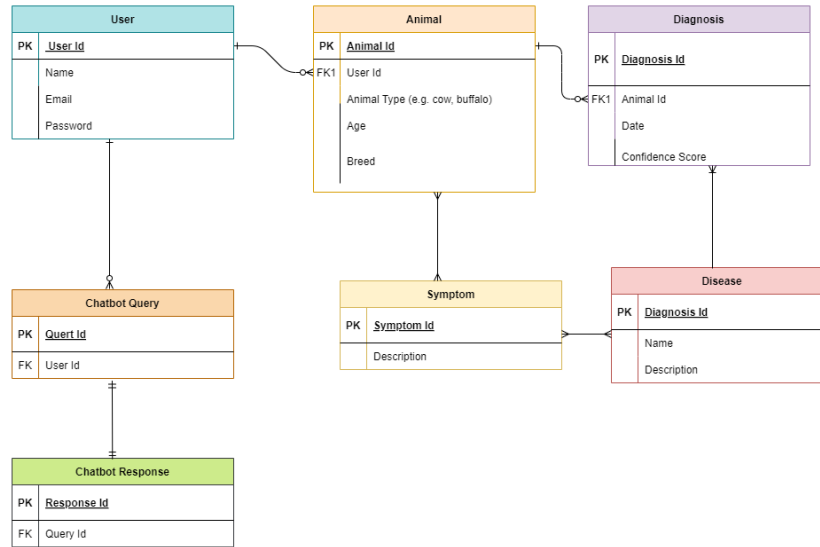


Figure 6: Entity Relationship Diagram illustrating the relationships between entities in the system.

7.7 Data Flow Diagram

The Data Flow Diagram (DFD) shows how data moves through the system, illustrating processes, data stores, and data flows. There are three levels of DFD: DFD0, DFD1, and DFD2.

7.7.1 DFD0

DFD0, also known as the context-level diagram, provides a high-level overview of the system, showing its main process and interactions with external entities.

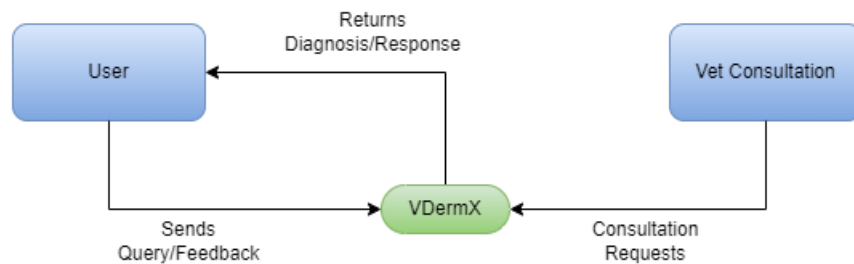


Figure 7: DFD0 - Context-level Data Flow Diagram showing the system's interactions with external entities.

7.7.2 DFD1

DFD1 expands the context diagram into more detailed processes and shows how the main process is divided into sub-processes.

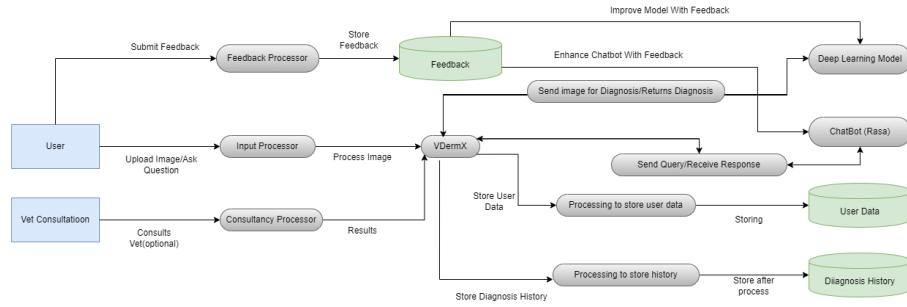


Figure 8: DFD1 - High-level Data Flow Diagram illustrating the breakdown of the system's main processes into sub-processes.

7.7.3 DFD2

DFD2 provides an even more detailed breakdown of the processes shown in DFD1, highlighting specific data flows within the sub-processes.

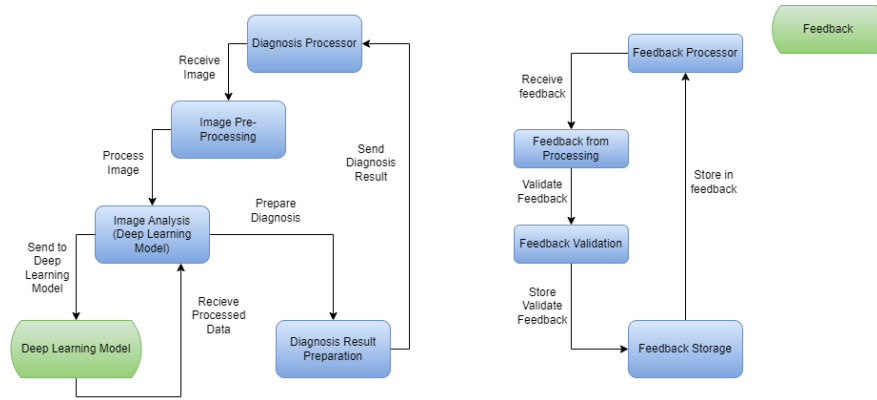


Figure 9: DFD2 - Detailed Data Flow Diagram showing data flows and interactions within sub-processes of the system.

7.8 Component Diagram

The component diagram illustrates the structure of the system's software components and their relationships.

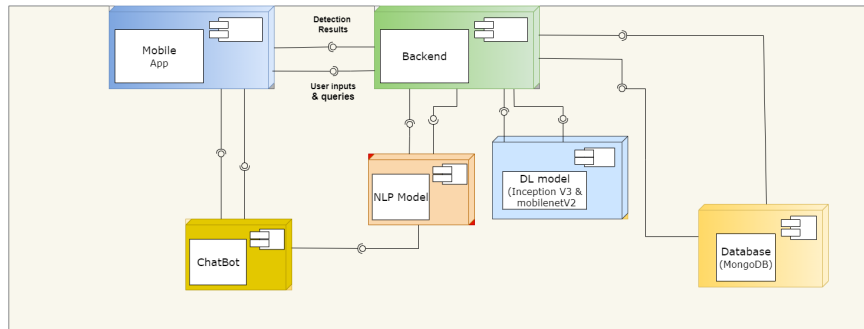


Figure 10: Component Diagram showing the software components and their interactions within the system.

7.9 Deployment Diagram

The deployment diagram shows the hardware and software environment in which the system operates, including nodes and the artifacts deployed on them.

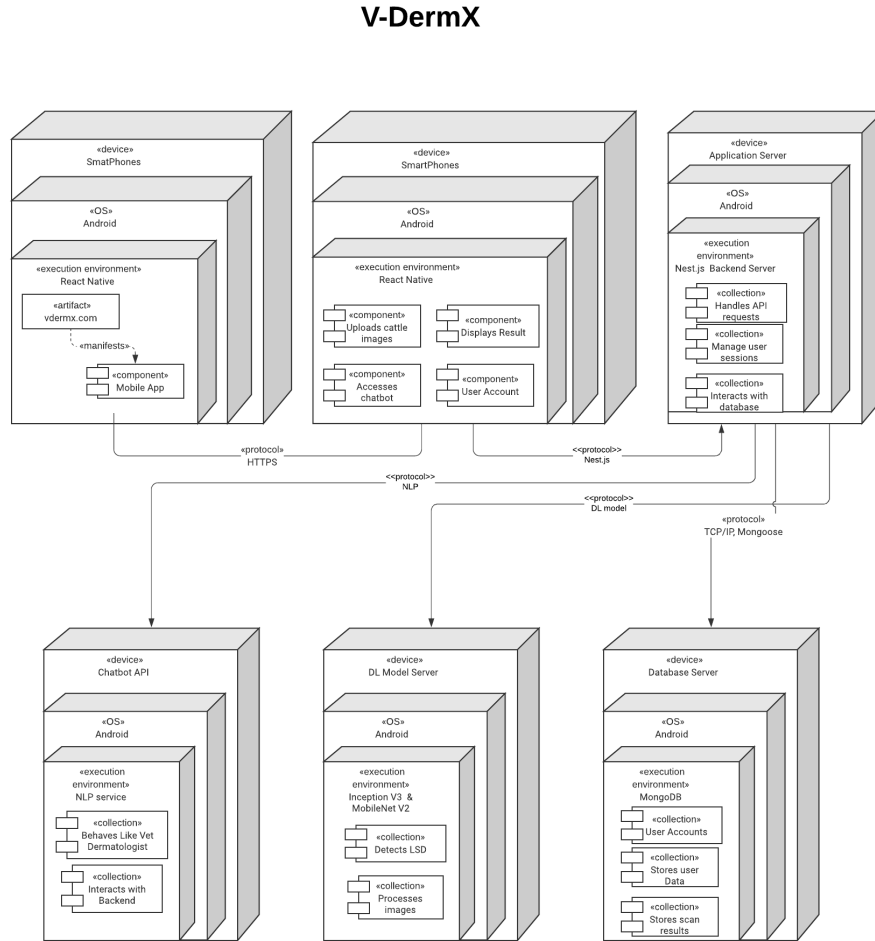


Figure 11: Deployment Diagram illustrating the physical deployment of software on the hardware infrastructure.

7.10 Context Diagram

The context diagram provides a high-level view of the system and how it interacts with external entities such as users or other systems.

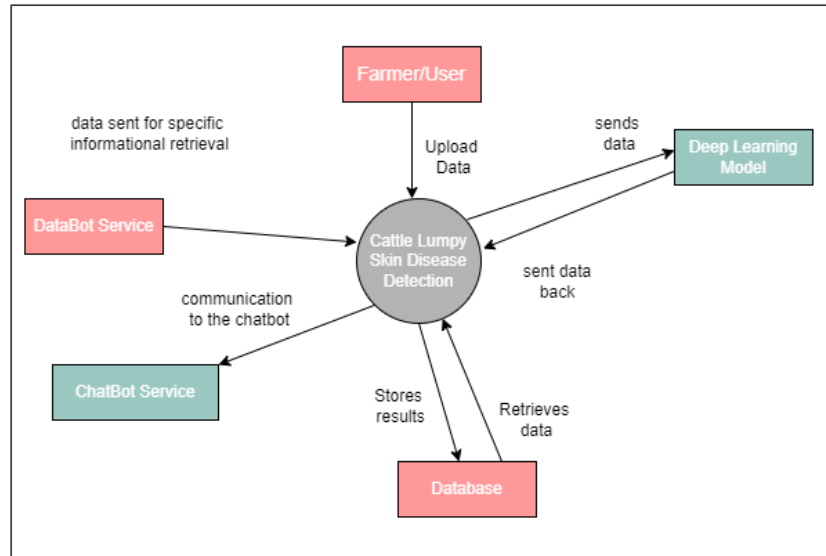


Figure 12: Context Diagram representing the interaction between the system and external entities.

8 Future Scope

8.1 Enhanced Diagnostic Capabilities

Future updates may integrate advanced machine learning techniques, improving the accuracy of lumpy skin disease detection and expanding the range of detectable skin conditions.

8.2 Artificial Intelligence

AI could further enhance the chatbot's capabilities, providing personalized recommendations for treatment and prevention based on user history and behavior patterns.

8.3 Data Analytics

Advanced analytics could enable the collection of insights from user interactions and diagnostic outcomes, helping to identify trends in animal health and inform preventive measures.

8.4 Mobile and Web Expansion

The platform could expand to include a dedicated web interface for veterinarians and animal health professionals, facilitating broader access to the app's features and data.

8.5 Collaboration with Veterinary Networks

The app could integrate with veterinary databases and networks to provide users with up-to-date information on treatment options, local veterinary services, and health guidelines.

8.6 User Community Features

Implementing community features could enable users to share experiences, tips, and advice, fostering a support network among pet owners and farmers.

8.7 Regulatory Compliance

The app will need to adapt to evolving regulations regarding data privacy, animal health standards, and veterinary practices, ensuring compliance in various regions.

8.8 Sustainability Practices

Future developments could focus on promoting sustainable farming and animal care practices, providing resources and recommendations that align with environmental goals.

8.9 Financial Integration

The app may explore partnerships with financial services to offer users financing options for veterinary care and treatments, enhancing accessibility to necessary services.

8.10 Augmented Reality (AR)

AR features could be introduced to help users visualize treatment options or provide guided instructions for at-home care.

9 Conclusion

The System Requirements Specification (SRS) for Vet Derm-X is a critical document that articulates the functional and non-functional requirements necessary for developing a robust application aimed at detecting and managing lumpy skin disease. This SRS serves as a solid foundation for the design, development, and testing phases of the project, ensuring that the final application meets the diverse needs of veterinarians, farmers, and pet owners.

First and foremost, the SRS emphasizes the importance of security and data integrity. Given the sensitive nature of health-related data, safeguarding user information through encryption, secure data storage, and access controls is paramount. By prioritizing these security measures, Vet Derm-X aims to build trust with its users while ensuring compliance with relevant regulations.

Moreover, the comprehensive coverage of the application's functionalities is a crucial aspect of this SRS. It delineates essential features such as image diagnosis, chatbot consultations, offline capabilities, and real-time notifications. This detailed outline enables developers to fully grasp the project scope, reducing the risk of misunderstandings during development and ensuring that the application delivers valuable and timely information to users. In conclusion, the System Requirements Specification for Vet Derm-X is an indispensable guide that lays the groundwork for the successful development and deployment of a sophisticated and secure application tailored for animal health management. It encapsulates vital elements of functionality, security, performance, and user experience, which are critical in the veterinary domain. A well-defined SRS mitigates ambiguity, minimizes risks, and acts as a roadmap for developers and stakeholders alike. By adhering to the requirements set forth in this document, Vet Derm-X can look forward to delivering a reliable, user-centric solution that not only addresses immediate challenges in animal health management but also adapts to future opportunities in the evolving landscape of veterinary care.

10 Appendices

10.1 Appendix A: Glossary of Terms

- **LSD:** Lumpy Skin Disease
- **AI:** Artificial Intelligence
- **ML:** Machine Learning
- **DL:** Deep Learning
- **CNN:** Convolutional Neural Network
- **InceptionV3:** A deep learning architecture used for image classification
- **MobileNetV2:** A lightweight neural network architecture
- **JWT:** JSON Web Token, used for secure authentication
- **Bcrypt:** A password-hashing function for secure password storage
- **RBAC:** Role-Based Access Control

10.2 Appendix B: API Specifications

The following table describes the APIs used in the V-DermX system:

API Name	Method	Description
Image Upload API	POST	Uploads animal images for diagnosis
Diagnosis API	GET	Retrieves the diagnosis of an image
Chatbot API	POST	Sends a user query to the chatbot and receives a response
Sync Data API	POST	Syncs offline data with the server when online

10.3 Appendix C: Detailed Algorithms

- **InceptionV3 Image Classification Algorithm:** Detailed architecture and layers used for image analysis.
- **MobileNetV2 Image Classification Algorithm:** Overview of the lightweight neural network for mobile applications.

11 References

- Rai, G., Naveen, Hussain, A., Kumar, A., Ansari, A., & Khanduja, N. (2021). A deep learning approach to detect lumpy skin disease in cows. In *Computer Networks, Big Data and IoT: Proceedings of ICCBI 2020* (pp. 369-377). Springer Singapore.
- Ujjwal, N., Singh, A., Jain, A. K., & Tiwari, R. G. (2022, October). Exploiting machine learning for lumpy skin disease occurrence detection. In *2022 10th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)(ICRITO)* (pp. 1-6). IEEE.
- Genemo, M. (2023). Detecting high-risk area for lumpy skin disease in cattle using deep learning feature. *Advances in Artificial Intelligence Research*, 3(1), 27-35.
- Muhammad Saqib, S., Iqbal, M., Tahar Ben Othman, M., Shahazad, T., Yasin Ghadi, Y., Al-Amro, S., & Mazhar, T. (2024). Lumpy skin disease diagnosis in cattle: A deep learning approach optimized with RMSProp and MobileNetV2. *Plos one*, 19(8), e0302862.
- Narayan, V., Awasthi, S., Fatima, N., Faiz, M., Bordoloi, D., Sandhu, R., & Srivastava, S. (2023, May). Severity of lumpy disease detection based on deep learning technique. In *2023 International Conference on Disruptive Technologies (ICDT)* (pp. 507-512). IEEE.
- Saha, D. K. (2024). An extensive investigation of convolutional neural network designs for the diagnosis of lumpy skin disease in dairy cows. *Heliyon*, 10(14).
- Kumar, A. H. (2023). Analysis of ChatGPT tool to assess the potential of its utility for academic writing in biomedical domain. *Biology, Engineering, Medicine and Science Reports*, 9(1), 24-30.
- Bocklisch, T., Faulkner, J., Pawlowski, N., & Nichol, A. (2017). Rasa: Open source language understanding and dialogue management. *arXiv preprint arXiv:1712.05181*.
- Mai, T. N. T., & Maxim, S. (2021). Enhancing Rasa NLU model for Vietnamese chatbot. *International Journal of Open Information Technologies*, 9(1), 31-36.
- Jiao, A. (2020, March). An intelligent chatbot system based on entity extraction using RASA NLU and neural network. In *Journal of physics: conference series* (Vol. 1487, No. 1, p. 012014). IOP Publishing.